CI/CD Integration Complete Documentation

1. Project: Cognantic Portal - Azure CI/CD Setup

- Overview

➢ This document outlines the complete CI/CD setup process for deploying a **.NET Backend (Azure Container Apps)** and **React/Vite Frontend (Azure Static Web Apps)** using GitHub Actions.

- Prerequisites

➢ Before starting, ensure you have:

Requirement	Details
Azure Subscription	Active subscription with Contributor access
GitHub Repository	Source code hosted on GitHub
Azure Resources	Container App + Static Web App + ACR
Local Setup	Git, Azure CLI, Docker (for testing)

- 📁 Step 1: Create Workflow File

File path: `**.github/workflows/deploy.yml**`

```yaml
name: Deploy Cognantic to Azure

on:
  push:
    branches:
      - main
    paths:
```

```yaml
    - 'Backend/**'
    - 'Frontend/**'
    - 'Dockerfile'
    - '.github/workflows/**'
  workflow_dispatch:

env:
  AZURE_CONTAINER_REGISTRY: "yourregistry.azurecr.io"
  CONTAINER_APP_NAME: "your-api"
  RESOURCE_GROUP: "your-resource-group"

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0

      - name: Restore & Build
        run: |
          dotnet restore Backend/API/API.csproj
          dotnet build Backend/API/API.csproj --configuration Release

      - name: Run Tests
        run: dotnet test Backend/API/API.csproj --no-build
        continue-on-error: true

      - name: Login to ACR
        uses: azure/docker-login@v1
        with:
          login-server: ${{ env.AZURE_CONTAINER_REGISTRY }}
          username: ${{ secrets.ACR_USERNAME }}
          password: ${{ secrets.ACR_PASSWORD }}

      - name: Build & Push Docker
        run: |
          docker build -t ${{ env.AZURE_CONTAINER_REGISTRY }}/api:${{ github.sha }} .
          docker push ${{ env.AZURE_CONTAINER_REGISTRY }}/api:${{ github.sha }}

      - name: Login to Azure
        uses: azure/login@v2
        with:
```

```
        creds: '{"clientId":"${{ secrets.AZURE_CLIENT_ID }}","clientSecret":"${{
secrets.AZURE_CLIENT_SECRET }}","subscriptionId":"${{
secrets.AZURE_SUBSCRIPTION_ID }}","tenantId":"${{ secrets.AZURE_TENANT_ID }}"}'

      - name: Deploy to Container App
        uses: azure/cli@v2
        with:
          inlineScript: |
            az containerapp update --name ${{ env.CONTAINER_APP_NAME }} \
              --resource-group ${{ env.RESOURCE_GROUP }} \
              --image ${{ env.AZURE_CONTAINER_REGISTRY }}/api:${{ github.sha }}

  frontend:
    runs-on: ubuntu-latest
    needs: backend
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install & Build
        run: |
          cd frontend
          npm ci
          npm run build
        env:
          VITE_API_URL: ${{ secrets.VITE_API_URL }}

      - name: Deploy to Static Web App
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${{ secrets.STATIC_WEB_APP_TOKEN }}
          repo_token: ${{ secrets.GITHUB_TOKEN }}
          action: "upload"
          app_location: "frontend"
          output_location: "dist"
```

- 🔐 Step 2: Add GitHub Secrets

Location: GitHub Repository → Settings → Secrets and variables → Actions

Secret Name	Description	Where to Get
`AZURE_CLIENT_ID`	Service Principal App ID	Azure AD → App Registrations
`AZURE_TENANT_ID`	Directory ID	Azure AD → Overview
`AZURE_SUBSCRIPTION_ID`	Subscription ID	Subscriptions blade
`AZURE_CLIENT_SECRET`	Service Principal Secret	Certificates & secrets
`ACR_USERNAME`	Container Registry username	ACR → Access keys
`ACR_PASSWORD`	Container Registry password	ACR → Access keys
`STATIC_WEB_APP_TOKEN`	Static Web App token	Static Web App → Management tokens
`VITE_API_URL`	Backend API URL	Your Container App URL

- 👤 Step 3: Create & Configure Service Principal

3.1 Create Service Principal using Azure CLI:

```bash
az ad sp create-for-rbac --name "github-cicd" --role Contributor --scope /subscriptions/SUBSCRIPTION_ID/resourceGroups/RESOURCE_GROUP
```

3.2 Save the output:

```json
{
  "appId": "CLIENT_ID",
  "password": "CLIENT_SECRET",
  "tenant": "TENANT_ID"
}
```

3.3 Add these values to GitHub Secrets (Step

- 🔑 Step 4: Assign Contributor Role (If Not Auto-Assigned)

If the Service Principal doesn't have permissions, run:

```bash
az role assignment create \
  --assignee CLIENT_ID \
  --role Contributor \
  --scope /subscriptions/SUBSCRIPTION_ID/resourceGroups/RESOURCE_GROUP
```

Verify in Azure Portal: Resource Group → Access Control (IAM) → Role assignments

---

- 🚀 Step 5: Test the CI/CD Pipeline

5.1 Push code to trigger deployment:

```bash
git add .
git commit -m "test: trigger CI/CD pipeline"
git push origin main
```

5.2 Monitor in GitHub:

1. Go to **Actions** tab
2. Click on the workflow run
3. Watch each job execute

5.3 Verify deployment:

```bash
# Check backend
curl https://your-api.azurecontainerapps.io/api/v1/health

# Check frontend
open https://your-static-app.azurestaticapps.net
```

- 📊 Step 6: Optional - PR Validation Workflow

File: `.github/workflows/pr-checks.yml`

```yaml
name: PR Validation

on:
  pull_request:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0

      - name: Build Backend
        run: dotnet build Backend/API/API.csproj

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Build Frontend
        run: |
          cd frontend
          npm ci
          npm run build
```

--

- 🦎 Common Issues & Solutions

Issue	Solution
`deployment_token was not provided`	Add `STATIC_WEB_APP_TOKEN` as GitHub secret
`No subscriptions found`	Assign Contributor role to Service Principal
`az login failed`	Use `creds` JSON format instead of individual params
`Invalid expression term '/'`	Check for syntax errors in Program.cs
`Node.js 20 deprecation`	Not urgent - update actions by Sept 2026
`Resource not accessible`	Check GitHub token permissions

✅ Success Checklist

After completing all steps, verify:

- [ ] GitHub Actions workflow runs on `push` to `main`
- [ ] Backend builds successfully
- [ ] Docker image pushes to ACR
- [ ] Container App updates with new image
- [ ] Frontend builds successfully
- [ ] Static Web App deploys new version
- [ ] Live site shows latest changes
- [ ] API endpoints respond correctly

- 📝 Quick Reference Commands

```bash
# Trigger deployment
git add . && git commit -m "deploy" && git push origin main

# Check workflow status
gh run list --workflow "deploy.yml"

# View logs
gh run watch

# Re-run failed workflow
gh run rerun RUN_ID
```

```

---

- 🎯 What This CI/CD Automates

| Step | Before (Manual) | After (Automatic) |
|------|----------------|------------------|
| Build backend | `dotnet build` | ✅ Auto |
| Run tests | `dotnet test` | ✅ Auto |
| Build Docker | `docker build` | ✅ Auto |
| Push to ACR | `docker push` | ✅ Auto |
| Deploy backend | `az containerapp update` | ✅ Auto |
| Build frontend | `npm run build` | ✅ Auto |
| Deploy frontend | Manual upload | ✅ Auto |

- 📞 Support & Resources

- [Azure Static Web Apps Documentation](https://docs.microsoft.com/en-us/azure/static-web-apps/)
- [Azure Container Apps Documentation](https://docs.microsoft.com/en-us/azure/container-apps/)
- [GitHub Actions Documentation](https://docs.github.com/en/actions)
- [Azure/login Action](https://github.com/Azure/login)

**Documentation Complete ✅
```